JURNAL PRAKTIKUM IOT

MODUL 4

KOMUNIKASI LONG RANGE (LoRa P2P & LoRaWAN)

Kelompok : .....

Hari/Shift : .....

Kode Asisten Pengajar : .....

Nama : .....

NIM : .....

PRAKTIKUM PEMROGRAMAN PERANGKAT IOT

FAKULTAS TEKNIK ELEKTRO

UNIVERSITAS TELKOM

2026

A. Rangkaian dan Sistem

Modul ini terdiri atas **dua bagian**. **Part A** memakai LoRa peer-to-peer dalam bentuk sebuah permainan: satu papan Beacon disembunyikan, dan tiap kelompok memburunya dengan mendengarkan kekuatan sinyalnya. **Part B** memakai LoRaWAN: papan yang sama berbicara ke gateway WM1302 melalui protokol yang mengatur kanalnya sendiri. Perbandingan keduanya inilah yang menjadi inti modul ini — pada Part A seluruh aturan jaringan kalian susun sendiri, sedangkan pada Part B aturan itu sudah disediakan oleh protokolnya.

A.A. Part A — Perburuan LoRa

Aturan permainan. Asisten menyembunyikan **satu** papan Beacon di suatu tempat di dalam jangkauan hotspot. Beacon itu memancar terus-menerus lima kali tiap detik. Tiap kelompok memegang satu papan Hunter yang **hanya mendengarkan**, lalu melaporkan kekuatan sinyal yang ditangkapnya ke papan skor melalui Wi-Fi. Semakin dekat kalian, semakin kuat sinyal yang terbaca. Kelompok pertama yang benar-benar memegang papan Beacon-lah pemenangnya.

1. Siapkan komponen berikut:

   - Cosmic LoRa Aurora V2 (1x per kelompok, sebagai Hunter)
   - Cosmic LoRa Aurora V2 (1x sekelas, sebagai Beacon)
   - Antena LoRa 920 MHz (1x per papan)
   - Orange Pi 4 PRO (1x sekelas, hotspot dan papan skor)
   - Power bank atau kabel USB panjang (1x per kelompok)

2. Berbeda dengan modul-modul sebelumnya, Part A ini **tidak memakai satu pun sensor maupun aktuator luar**: yang diukur adalah gelombang radionya sendiri. Karena itu tidak ada kabel yang perlu dirangkai — cukup pasang antena, lalu nyalakan papannya. Tabel di bawah mencatat pin radio yang sudah tersambung di dalam papan dan dipakai oleh program.

Skema topologi Perburuan LoRa Modul 4 Part A (menyusul)

Pin radio RFM95W pada Cosmic LoRa Aurora V2			
Sinyal	**Pin**	**Sinyal**	**Pin**
RFM (NSS)	GPIO15	RFM (DIO0)	GPIO27
RFM (RST)	GPIO0	RFM (SCK)	GPIO18
RFM (MISO)	GPIO19	RFM (MOSI)	GPIO23
RFM (EN)	GPIO32		

*Catatan: pasanglah antena 920 MHz **sebelum** kabel USB dicolokkan. Memancar tanpa antena dapat merusak radio secara permanen, dan hal itu berlaku pada papan Beacon maupun papan Hunter. Pin GPIO 15, 0, 27, 32, 18, 19, dan 23 sudah dipakai oleh modul radio, sehingga tidak boleh dipakai untuk keperluan lain. Hunter tidak memerlukan pembagi tegangan karena tidak ada sensor yang tersambung padanya.*

Pembagian berkas per kelompok. Tiap kelompok memiliki foldernya sendiri yang sudah berisi nomor kelompoknya. Salin folder milik kelompok kalian, lalu unggah `hunter.ino` ke papan kalian — tidak ada baris yang perlu disunting. Berkas `beacon.ino` di dalam folder itu sama untuk semua kelompok dan hanya diunggah oleh asisten ke papan Beacon.

Kelompok	Folder program	Kelompok	Folder program
1	`modul4/grup1/`	5	`modul4/grup5/`
2	`modul4/grup2/`	6	`modul4/grup6/`
3	`modul4/grup3/`	7	`modul4/grup7/`
4	`modul4/grup4/`	–	–

3. Sebelum menjalankan program, siapkan lingkungan kerja kalian dengan urutan berikut. Centanglah setiap langkah yang telah kalian selesaikan!

   - ☐ Memasang antena 920 MHz pada papan **sebelum** kabel USB dicolokkan
   - ☐ Memasang pustaka `LoRa` karya Sandeep Mistry melalui Library Manager Arduino IDE
   - ☐ Memasang pustaka `WebSockets` karya Markus Sattler melalui Library Manager Arduino IDE
   - ☐ Memilih board ESP32 Dev Module pada Arduino IDE
   - ☐ Memastikan Orange Pi sudah menyalakan hotspot `IRIS-HUNT` dan papan skornya
   - ☐ Menyalin folder milik kelompok kalian, lalu mengunggah `hunter.ino` tanpa menyunting apa pun
   - ☐ Memastikan Serial Monitor mencetak alamat IP dan jam hasil sinkronisasi
   - ☐ Membuka papan skor pada `http://192.168.4.1:8080`, memasukkan NIM masing-masing, dan memastikan nama kelompok kalian yang muncul
   - ☐ Mengkalibrasi acuan jarak: pegang satu Hunter tepat 1 meter dari Beacon, lalu tekan **Kalibrasi 1 m** pada kartunya
   - ☐ Menyalakan **Detak jarak** pada ponsel pemandu kelompok, dan menyetel batas tidur layarnya ke 5 menit sebelum berangkat

   *Catatan regulasi: pita yang diizinkan di Indonesia adalah 920–923 MHz (AS923-2), sehingga nilai frekuensi yang dipakai pada praktikum ini adalah `920E6`. Nilai `433E6`, `866E6`, dan `915E6` adalah pita untuk wilayah lain dan **tidak boleh** dipakai di sini. Beacon memancar pada SF7 dengan lebar pita 250 kHz, sehingga satu paket hanya sekitar 20 milidetik di udara; dengan jeda 200 milidetik, kanal terpakai kira-kira sepersepuluh waktu saja. Laju sepadat ini hanya pantas untuk sesi praktikum yang singkat dan berdaya kecil; perangkat AS923 yang dipasang permanen justru dijaga jauh di bawah angka itu.*

   *Catatan daya pancar: daya beacon disetel menurut **ukuran arena**, bukan dibesarkan sebisanya. Pada 20 dBm seluruh gedung terbaca sama kuatnya dan beacon justru mustahil ditemukan; tetapi pada daya yang terlalu kecil, kelompok terjauh tidak mendengar apa pun sejak awal dan permainannya juga mati. Nilai bawaan `BEACON_TX_POWER` adalah **10 dBm**, yang menurut anggaran tautan pada SF7 masih terdengar*

sampai sekitar satu kilometer di ruang terbuka dan beberapa ratus meter di dalam gedung bertembok. Pakailah 2 untuk satu ruangan, 10 untuk satu lantai gedung, dan 14 untuk gedung besar.

4. Lengkapi kode program di bawah ini dengan mengisikan bagian-bagian yang rumpang. Selanjutnya (atau sambil melengkapi), ketiklah kembali bagian-bagian rumpang yang telah diisi pada aplikasi VS Code/Arduino IDE yang telah disiapkan oleh asisten jaga kalian masing-masing!

   Bagian rumpang ditandai dengan garis bawah `_____` dan diberi nomor `(1)` sampai `(6)` pada komentar di sebelah kanannya. Gantilah setiap garis bawah tersebut dengan kode yang benar sesuai petunjuk pada komentarnya. Keempat nilai radio — frekuensi, spreading factor, lebar pita, dan sync word — harus sama persis antara Hunter dan Beacon; bila salah satu berbeda, paketnya tidak akan pernah terdengar sama sekali.

Hunter Source Code — Perburuan LoRa (Cosmic LoRa Aurora V2)

```
 */

#include <WiFi.h>
#include <SPI.h>
#include <________>  // (1) Header pustaka LoRa peer-to-peer oleh Sandeep Mistry
#include <WebSocketsClient.h>
#include <time.h>
#include <sys/time.h>
…
void connectToWiFi() {
  Serial.print("Menyambung ke hotspot Orange Pi");

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, ________);  // (2) Variabel kata sandi Wi-Fi

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
…
    Serial.println("Radio RFM95W belum terdeteksi, mencoba lagi...");
    delay(500);
  }

  LoRa.setSpreadingFactor(________);  // (3) Konstanta spreading factor
  LoRa.setSignalBandwidth(LORA_BANDWIDTH);
  LoRa.setSyncWord(________);  // (4) Konstanta penanda pasangan, sama dengan beacon
}

void setup() {
  Serial.begin(115200);
…
  }

  String receivedAt = wallClockTimestamp();

  int rssi = LoRa.________();  // (5) Metode pembaca kekuatan sinyal paket
  float snr = LoRa.packetSnr();

  // Paket beacon berisi "<no. paket> <waktu kirim>", dipisahkan satu spasi.
  int space = beaconLine.indexOf(' ');
…
            GROUP_ID, seq.c_str(), rssi, smoothRssi(rssi), snr,
            sentAt.c_str(), receivedAt.c_str(),
            WiFi.localIP().toString().c_str());

  webSocket.________(payload);  // (6) Metode pengirim pesan teks

  Serial.printf("[%s] Paket #%s | RSSI %d dBm (halus %.1f) | SNR %.1f dB\n",
                receivedAt.c_str(), seq.c_str(), rssi, smoothed, snr);
}
```

Beacon Source Code — Perburuan LoRa (Cosmic LoRa Aurora V2)

```
 */

#include <WiFi.h>
#include <SPI.h>
#include <________>  // (1) Header pustaka LoRa peer-to-peer oleh Sandeep Mistry
#include <WebSocketsClient.h>
#include <time.h>
#include <sys/time.h>
…
}

void syncClock() {
  Serial.print("Menyelaraskan jam ke Orange Pi");
  configTime(GMT_OFFSET_SEC, DST_OFFSET_SEC, ________);  // (2) Alamat server NTP

  struct tm timeinfo;
  while (!getLocalTime(&timeinfo, 1000)) {
    Serial.print(".");
…
    delay(500);
  }

  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);
  LoRa.setSignalBandwidth(________);  // (3) Konstanta lebar pita radio
  LoRa.setSyncWord(LORA_SYNC_WORD);
  LoRa.setTxPower(________);  // (4) Konstanta daya pancar beacon
}

void setup() {
  Serial.begin(115200);
…
  String sentAt = wallClockTimestamp();
  String payload = String(seq) + " " + sentAt;

  LoRa.beginPacket();
  LoRa.print(________);  // (5) Variabel isi paket yang disusun di atas
  LoRa.________();  // (6) Metode pemancar paket yang sudah disiapkan

  // Papan skor perlu tahu nomor paket yang benar-benar dipancarkan, supaya
  // paket yang tidak sampai ke sebuah hunter dapat dihitung sebagai hilang.
  char report[64];
```

5. Catatlah konfigurasi yang kalian pakai pada tabel berikut ini! Keempat nilai radio yang pertama harus sama persis antara Hunter dan Beacon agar paket dapat terbaca.

Parameter	Nilai yang dipakai
Frekuensi kerja (`LORA_FREQUENCY`)	
Spreading factor (`LORA_SPREADING_FACTOR`)	
Lebar pita (`LORA_BANDWIDTH`)	
Sync word (`LORA_SYNC_WORD`)	
Daya pancar beacon (`BEACON_TX_POWER`)	
Jeda pancar beacon (`BEACON_PERIOD_MS`)	
Nama dan alamat hotspot (`WIFI_SSID` / `SERVER_HOST`)	
Nomor kelompok (`GROUP_ID`) dan alamat IP papan kalian	

A.B. Part B — LoRaWAN melalui Gateway WM1302

6. Siapkan komponen berikut:

- Cosmic LoRa Aurora V2 (papan Part A) (1x per kelompok)
- Antena LoRa 920 MHz (1x per kelompok)
- Sensor ultrasonik HC-SR04 (1x per kelompok)

- Resistor 1 kΩ dan 2 kΩ (1x per kelompok, pembagi ECHO)
- Breadboard (1x per kelompok)
- Gateway Wio-WM1302 + hat (1x sekelas)
- Orange Pi 4 PRO (ChirpStack) (1x sekelas)
- Kabel/Jumper (disesuaikan)

7. Berbeda dengan Part A yang tidak memakai sensor sama sekali, Part B kembali mengirimkan pembacaan HC-SR04. Rangkailah sensornya seperti tabel di bawah, lalu perhatikan bahwa pustaka LMIC juga memakai satu pin radio tambahan.

Sensor HC-SR04 (dirangkai sendiri)		**Radio RFM95W (sudah di dalam papan)**	
Komponen	**Pin**	**Sinyal**	**Pin**
US (VCC)	5V	RFM (NSS)	GPIO15
US (TRIG)	GPIO16	RFM (RST)	GPIO0
US (ECHO) ⟶ R1K	GPIO17	RFM (DIO0)	GPIO27
R1K ⟶ R2K	GND	RFM (DIO1)	GPIO2
US (GND)	GND	RFM (EN/VMODEM)	GPIO32

Catatan: pin ECHO pada HC-SR04 mengeluarkan 5 V, sehingga wajib melewati pembagi tegangan 1 kΩ/2 kΩ sebelum masuk ke GPIO17; tanpa pembagi ini pin ESP32 dapat rusak. Pada sisi radio, perbedaannya dengan Part A hanya satu: LoRa peer-to-peer cukup memakai DIO0, sedangkan LoRaWAN juga membutuhkan DIO1 untuk menandai jendela penerimaan setelah paket dipancarkan. Pin DIO1 sudah tersambung ke GPIO2 pada papan Cosmic, sehingga tidak ada kabel yang perlu dipindahkan.

8. Sebelum menjalankan program, siapkan lingkungan kerja kalian dengan urutan berikut. Centanglah setiap langkah yang telah kalian selesaikan!

   - ☐ Memasang pustaka `MCCI LoRaWAN LMIC library` melalui Library Manager Arduino IDE
   - ☐ Memastikan berkas `lmic_project_config.h` memakai pita `CFG_as923` dan `CFG_sx1276_radio`
   - ☐ Mendaftarkan papan kelompok kalian pada ChirpStack dengan DEVEUI dari tabel di atas
   - ☐ Menyalin APPKEY dari halaman device ChirpStack ke dalam program node
   - ☐ Memastikan gateway WM1302 sudah berstatus *online* pada ChirpStack
   - ☐ Menyinkronkan jam papan node dari PC: `.\scripts\send_time.ps1 -Port COM_A`
   - ☐ Menjalankan program pemantau gateway pada Orange Pi 4 PRO

   Catatan: pada Part A kalian harus membagi jatah detik sendiri agar delapan papan tidak saling menabrak. Pada Part B pembagian itu tidak diperlukan, karena gateway WM1302 mendengarkan delapan kanal sekaligus dan protokol LoRaWAN yang mengatur kapan tiap node boleh memancar. Inilah alasan keberadaan sebuah gateway.

9. Lengkapi kode program di bawah ini dengan mengisikan bagian-bagian yang rumpang, sama seperti pada Part A!

Node Source Code — LoRaWAN OTAA (Cosmic LoRa Aurora V2)

```
11    * Isi setiap bagian rumpang; nomor (1) sampai (6) pada komentar di sebelahnya
12    * adalah petunjuknya.
```

```
 */

#include <________>  // (1) Header utama pustaka MCCI LoRaWAN LMIC
#include <hal/hal.h>
#include <SPI.h>
#include <time.h>
…
// terdaftar sendiri di ChirpStack. APPKEY disalin dari halaman device
// pada ChirpStack, ditulis dalam urutan MSB.
static const u1_t PROGMEM APPEUI[8]  = { 0, 0, 0, 0, 0, 0, 0, 0 };
static const u1_t PROGMEM DEVEUI[8]  = { GROUP_ID, 0x00, 0x00, 0x00, 0x00, 0x00, 0x53,
0x49 };
static const u1_t PROGMEM APPKEY[16] = { ________ };  // (2) Enam belas byte APPKEY dari
ChirpStack

void os_getArtEui(u1_t *buf) { memcpy_P(buf, APPEUI, 8); }
void os_getDevEui(u1_t *buf) { memcpy_P(buf, DEVEUI, 8); }
void os_getDevKey(u1_t *buf) { memcpy_P(buf, APPKEY, 16); }
…
const lmic_pinmap lmic_pins = {
  .nss  = LORA_NSS,
  .rxtx = LMIC_UNUSED_PIN,
  .rst  = LORA_RST,
  .dio  = { LORA_DIO0, ________, LMIC_UNUSED_PIN },  // (3) Konstanta pin DIO1
};

static osjob_t sendjob;
…
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = ________(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);  // (4) Fungsi
pengukur lebar pulsa

  if (duration == 0) {
    return -1;
  }
…
    payload[0] = GROUP_ID;
    payload[1] = (uint8_t)(distance >> 8);
    payload[2] = (uint8_t)(distance & 0xFF);

    ________(APP_PORT, payload, sizeof(payload), 0);  // (5) Fungsi LMIC penjadwal
pengiriman data

    Serial.printf("[%s] Kelompok %d mengirim jarak %d cm ke gateway.\n",
                  formatTimestamp(millis()).c_str(), GROUP_ID, distance);
  }
…
}

void loop() {
  // Seluruh penjadwalan LoRaWAN dikerjakan oleh fungsi ini.
  ________();  // (6) Fungsi utama penjalan antrean LMIC
}
```

Gateway Monitor Source Code — ChirpStack MQTT (Orange Pi 4 PRO)

```
MQTT_PORT = 1883

# Topik uplink ChirpStack. Tanda + berarti application id dan device id
# mana pun, sehingga satu skrip memantau semua kelompok sekaligus.
UPLINK_TOPIC = ________  # (1) Topik uplink ChirpStack, memakai + untuk semua application
dan device


def timestamp() -> str:
    """Mengembalikan jam dinding saat ini dalam format HH:MM:SS:MS."""
…
    return group, distance
```

```python
def on_connect(client, userdata, flags, reason_code, properties=None) -> None:
    client._________(UPLINK_TOPIC)  # (2) Metode pendaftar langganan topik
    print(f"[{timestamp()}] Terhubung ke broker ChirpStack, menunggu uplink...")


def on_message(client, userdata, msg) -> None:
    received_at = timestamp()

    event = _________(msg.payload)  # (3) Fungsi pengurai teks JSON menjadi dict

    # ChirpStack mengirim payload dalam bentuk base64, jadi harus didekode dahulu.
    raw = base64.b64decode(event["data"])
    group, distance = decode_payload(raw)

    info = event.get("rxInfo", [{}])[0]
    rssi = info.get("rssi", 0)
    snr = info.get("_________", 0)  # (4) Nama field rasio sinyal terhadap derau pada rxInfo

    print(f"[{received_at}] Kelompok {group} | jarak {distance} cm | RSSI: {rssi} dBm | SNR:
{snr} dB")
…
def main() -> None:
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect
    client.on_message = _________  # (5) Fungsi penangan pesan yang baru masuk

    client.connect(MQTT_HOST, MQTT_PORT, 60)
    client._________()  # (6) Metode penjalan loop MQTT tanpa batas waktu


if __name__ == "__main__":
    main()
```

10. Catatlah hasil pengamatan Part B pada tabel berikut! Waktu kirim diambil dari Serial Monitor node, sedangkan waktu terima, RSSI, dan SNR diambil dari keluaran program pemantau pada Orange Pi.

Percobaan ke-…	Jarak **[cm]**	RSSI **[dBm]**	SNR **[dB]**	Waktu kirim *(Node)* **[HH:MM:SS:MS]**	Waktu terima *(Gateway)* **[HH:MM:SS:MS]**	Delay satu arah **[ms]**
1.						
2.						
3.						
4.						
5.						
Rata-rata delay (ms)						

B. Analisis

1. Gambarkanlah alur komunikasi yang diterapkan pada sistem yang telah kalian kerjakan!

2. Selanjutnya akan dilakukan percobaan respons waktu pengiriman paket dari **Beacon** menuju **Hunter** kelompok kalian. Bukalah Serial Monitor papan Hunter; tiap paket yang tertangkap dicetak lengkap dengan nomor paket, RSSI mentah, RSSI hasil penghalusan, dan SNR-nya. **Waktu kirim** dibawa oleh paketnya sendiri dari Beacon, sedangkan **waktu terima** dicatat Hunter saat paket itu tiba, sehingga keduanya dapat dibaca dari satu Serial Monitor yang sama. Nomor paket dan nilai yang sama juga muncul pada papan skor di `http://192.168.4.1:8080`. Catatlah:
   - **LIMA (5)** baris data: nomor paket, RSSI, SNR, waktu kirim, dan waktu terima

   Kedua waktu tersebut dicatat dalam format **HH:MM:SS:MS**. Hitunglah **delay satu arah** dengan mengurangkan waktu terima terhadap waktu kirim. Modul ini tidak memiliki kolom round-trip karena Hunter tidak pernah memancar sama sekali.

   *Catatan: Beacon maupun Hunter menyelaraskan jamnya melalui NTP ke **Orange Pi yang sama**, bukan ke Internet — hotspot praktikum memang tidak memiliki jalur keluar. Karena keduanya berangkat dari satu sumber waktu, selisih keduanya sahih dan boleh dikurangkan walaupun jam mutlak Orange Pi sendiri mungkin meleset beberapa detik dari waktu sebenarnya. Bila sebuah papan gagal menyelaraskan jamnya, timestamp-nya **bukan** jam dinding dan baris itu harus dikosongkan. Nilai RSSI dan SNR hanya dapat diukur pada sisi penerima, sehingga seluruhnya berasal dari Hunter.*

Percobaan ke-…	No. paket	RSSI **[dBm]**	SNR **[dB]**	Waktu kirim *(Beacon)* **[HH:MM:SS:MS]**	Waktu terima *(Hunter)* **[HH:MM:SS:MS]**	Delay satu arah **[ms]**
1.						
2.						
3.						
4.						
5.						
Rata-rata delay satu arah (ms)						

3. Terakhir, ujilah seberapa jujur RSSI sebagai penunjuk jarak. Papan skor menampilkan **perkiraan jarak dalam meter**, tetapi angka itu dihitung dari RSSI memakai model *log-distance* — bukan diukur. Letakkan papan Hunter pada tiga jarak yang kalian ukur sendiri dengan meteran, lalu catat RSSI halus

dan perkiraan papan skor untuk tiap posisi. Cobalah satu di antaranya dengan tembok atau badan kalian menghalangi garis pandang ke Beacon, dan bandingkan selisihnya.

Posisi	Jarak sebenarnya *(meteran)* **[m]**	RSSI halus **[dBm]**	Perkiraan papan skor **[m]**	Ada penghalang? *(ya / tidak)*
Dekat				
Sedang				
Jauh				

Catatan: perkiraan meter itu berasal dari rumus

$$d = 10^{\frac{P_{1\mathrm{m}} - \mathrm{RSSI}}{10n}}$$

*dengan $P_{1\mathrm{m}}$ adalah RSSI pada jarak satu meter (itulah yang kalian kalibrasi tadi) dan n adalah eksponen path loss, di sini 2,0 karena perburuan dimainkan di lapangan terbuka; di dalam gedung bersekat nilainya 2,5–3,5. Rumus ini mengandaikan sinyal melemah mulus terhadap jarak — andaian yang langsung runtuh begitu ada tembok. Karena itu papan skor selalu menampilkan rentangnya, bukan satu angka tunggal. Perhatikan pula bahwa rentang itu **melebar sebanding dengan jaraknya sendiri**: ketidakpastian ± 6 dB berarti "10 m" sebenarnya 6–17 m, sedangkan "800 m" sebenarnya 460–1390 m. Jadi RSSI berguna untuk memandu langkah ketika sudah dekat, dan hampir tidak berguna untuk menyatakan jarak ketika masih jauh.*

4. Tuliskanlah **analisis** kalian pada sistem yang telah kalian eksperimentasikan sebelumnya pada bagian pengisian berikut ini!

(Berikan juga **hubungan/korelasinya** dengan menerapkan **metodologi komunikasi** yang sedang kalian pelajari pada modul ini; tidak harus penuh!)

C. Saran dan Masukan

1. Asisten

2. Laboratorium